Unsupervised Learning on Handwritten Digits

Siti Raudhah Abas

November 2025

Introduction

This report analyses a dataset of 3,000 handwritten digits (5, 6, and 7) to explore unsupervised learning techniques. The high-dimensional pixel data ($28 \times 28 = 784$ dimensions) is processed to extract meaningful patterns without using true labels. The analysis proceeds in three tasks:

- **PCA**: Reducing dimensionality to capture the most important variations in the images.
- **Clustering**: Comparing K-means and Hierarchical clustering methods to identify digit groups.
- **Reconstruction**: Visually validating the PCA features by reversing the transformation.

1 Principal Component Analysis (PCA)

In this task, the goal is to reduce the 784 pixel dimensions of each handwritten image down to a smaller number of meaningful features. PCA does this by finding new directions (principal components) that capture the strongest patterns in the data. We keep the first 20 components, which explain most of the important variation in the digit images.

1.1 Reading and Preparing the Data

Each handwritten digit (5, 6, or 7) is represented by a vector of 784 grayscale pixel values. We load the dataset, separate the image matrix, and store the digit labels.

```
# Read the full data matrix
read_mat <- as.matrix(read.table("../data/digit.txt", sep=","))

# Separate pixel values and the digit labels
image_mat <- read_mat[, 2:785] # 784 pixel values
label <- read_mat[, 1] # digit label: 5, 6 or 7

dim(image_mat)
```

```
## [1] 3000  784
```

This confirms that the dataset has 3000 images and 784 pixel dimensions.

1.2 Performing PCA

```r
# Perform PCA
pca_out <- prcomp(image_mat)

# Extract the first 20 principal components
pca_scores_20 <- pca_out$x[, 1:20]
```

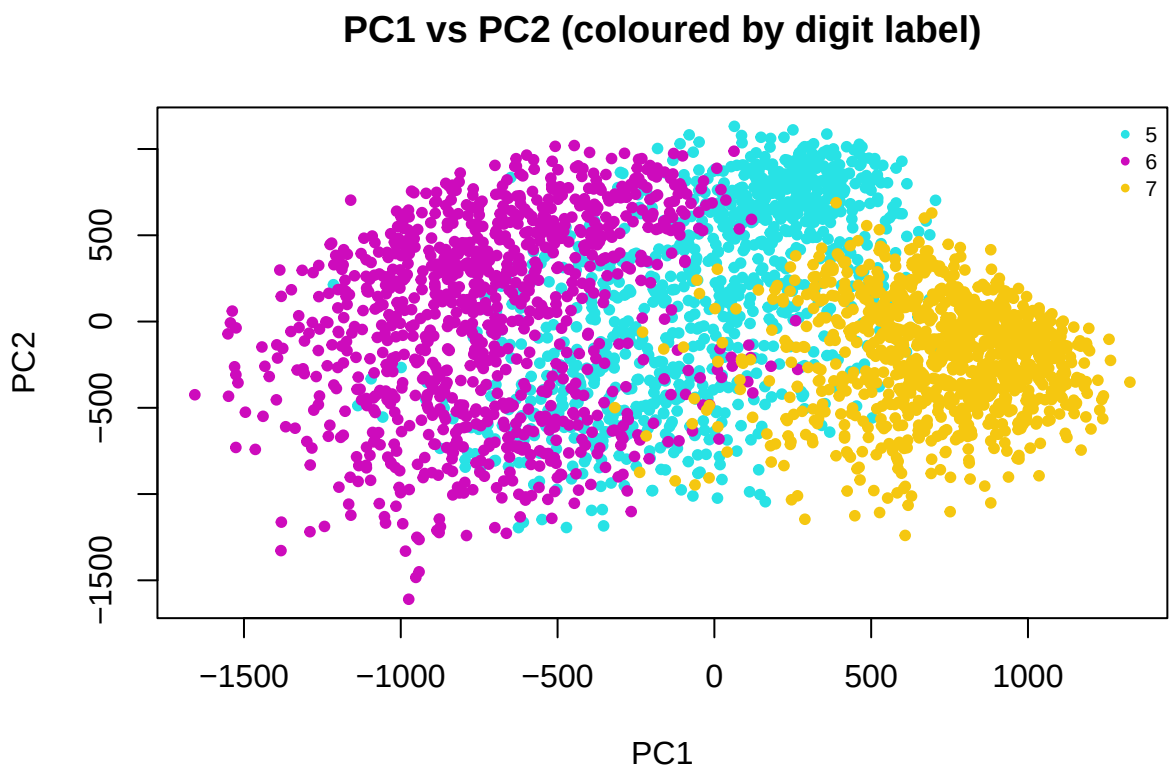
The resulting matrix pca_scores_20 is 3000 x 20, representing each digit in a much smaller feature space.

1.3 Plotting PC1 VS PC2

The first two principal components often reveal the most visible separation between groups. We colour the points using the true digit labels.

```r
plot(pca_scores_20[,1], pca_scores_20[,2],
     col = label,
     pch = 20,
     xlab = "PC1",
     ylab = "PC2",
     main = "PC1 vs PC2 (coloured by digit label)")

legend("topright",
       legend = c("5","6","7"),
       col = c(5,6,7),
       pch = 20,
       cex = 0.7,
       pt.cex = 0.7,
       bty = "n")
```



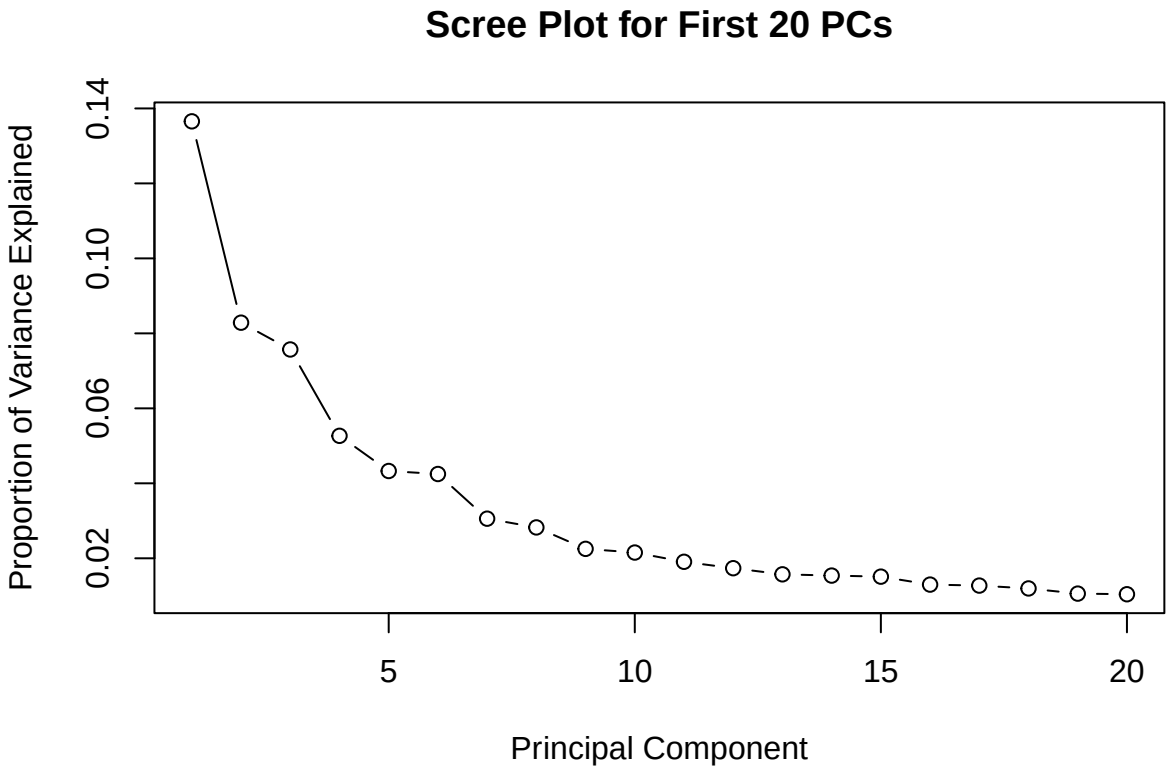
Interpretation: Digit clusters begin to form distinct regions in the PC1-PC2 plane. This suggests that the principal components capture meaningful shape differences between digits 5, 6, and 7.

1.4 Scree Plot (Variance Explained)

To check how many components should be kept, we look at the variance explained by each component.

```
# Variance explained by each principal component
var_explained <- pca_out$sdev^2 / sum(pca_out$sdev^2)

# Scree plot for the first 20 PCs
plot(1:20, var_explained[1:20], type = "b",
     xlab = "Principal Component",
     ylab = "Proportion of Variance Explained",
     main = "Scree Plot for First 20 PCs")
```



Interpretation: The first few components explain a large proportion of the total variation. After around 15-20 components, the variance contribution becomes very small. This validates the choice of reducing from 784 to 20 dimensions.

2 Clustering Analysis

```r
# Load and prepare data
read_mat <- as.matrix(read.table("../data/digit.txt", sep=","))

image_mat <- read_mat[, 2:785]
label <- read_mat[, 1]
true_label <- as.numeric(factor(label))

# PCA (first 20 components)
pca_out <- prcomp(image_mat)
pca20 <- pca_out$x[, 1:20]
```

Cluster numbers are arbitrary: cluster 1 is not necessarily digit 5. Before counting errors, each cluster is relabelled with the digit that gives the fewest misclassifications (trying all $3! = 6$ possible matchings).

```r
# Relabel clusters so they line up with true_label (1 = digit 5, 2 = 6, 3 = 7)
perms <- list(c(1,2,3), c(1,3,2), c(2,1,3), c(2,3,1), c(3,1,2), c(3,2,1))

match_clusters <- function(cl) {
  errors <- sapply(perms, function(p) sum(p[cl] != true_label))
  perms[[which.min(errors)]][cl]
}
```

2.1 K-means and Hierarchical Clustering

2.1.1 K-means Clustering (K = 3)

K-means is used here to group the digits into three clusters based on their PCA-reduced features. Since PCA already captures the main structure of variation, applying K-means on 20 principal components allows the algorithm to group observations using the most meaningful directions of change. We set centers $= 3$ because the dataset contains digits 5, 6, and 7. The *nstart* $= 25$ option ensures a stable solution by running K-means multiple times and selecting the best clustering.

```r
set.seed(1)
km_out <- kmeans(pca20, centers = 3, nstart = 25)
km_cluster <- km_out$cluster

# Confusion table
k_tab <- table(True_Digit = label, KMeans_Cluster = km_cluster)
knitr::kable(k_tab, caption = "Confusion Matrix: K-means Clustering")
```

Table 1: Confusion Matrix: K-means Clustering

	1	2	3
5	969	19	12
6	153	839	8
7	42	4	954

```r
# Misclassifications after matching clusters to digits
mis_kmeans <- sum(match_clusters(km_cluster) != true_label)
mis_kmeans
```

```
## [1] 238
```

The confusion table shows how well the three K-means clusters align with the actual digits. Most observations for each digit are grouped into a consistent cluster, although there is noticeable mixing between digits 5 and 6. This is expected because K-means forces spherical clusters and can struggle when class boundaries overlap. After matching each cluster to its digit, K-means misclassifies 238 of the 3000 images (7.9% error).

2.1.2 Hierarchical Clustering

To compare clustering behaviour, we apply four hierarchical clustering methods: single, complete, average, and Ward.D2 linkage. Each method defines cluster similarity differently, so their performance may vary. We compute a Euclidean distance matrix from the PCA-reduced data and run hierarchical clustering using each linkage rule. The resulting cluster trees are then cut into three groups to correspond to the three digit classes.

```r
dmat <- dist(pca20)

hc_single   <- hclust(dmat, method = "single")
hc_complete <- hclust(dmat, method = "complete")
hc_average  <- hclust(dmat, method = "average")
hc_ward     <- hclust(dmat, method = "ward.D2")

# Cut into 3 clusters
cut_single   <- cutree(hc_single,   k = 3)
cut_complete <- cutree(hc_complete, k = 3)
cut_average  <- cutree(hc_average,  k = 3)
cut_ward     <- cutree(hc_ward,     k = 3)
```

Misclassification Tables

```r
# Calculate errors (after matching clusters to digits)
mis_single   <- sum(match_clusters(cut_single) != true_label)
mis_complete <- sum(match_clusters(cut_complete) != true_label)
mis_average  <- sum(match_clusters(cut_average) != true_label)
mis_ward     <- sum(match_clusters(cut_ward) != true_label)

# Create a dataframe to compare the methods side-by-side
mis_all <- c(mis_kmeans, mis_single, mis_complete, mis_average, mis_ward)
comparison_df <- data.frame(
  Method = c("K-means", "Single Linkage", "Complete Linkage", "Average Linkage", "Ward.D2"),
  Misclassification_Count = mis_all,
  Error_Rate = mis_all / 3000,
  Accuracy = 1 - mis_all / 3000
)

knitr::kable(comparison_df,
             caption = "Comparison of Clustering Performance",
             digits = 3)
```

Table 2: Comparison of Clustering Performance

Method	Misclassification_Count	Error_Rate	Accuracy
K-means	238	0.079	0.921
Single Linkage	1998	0.666	0.334
Complete Linkage	1289	0.430	0.570
Average Linkage	1002	0.334	0.666
Ward.D2	65	0.022	0.978

Single Linkage Result: Single linkage performs poorly because it suffers from the "chaining" effect, where clusters are joined by long thin chains of points. This leads to very mixed clusters and a high misclassification count.

Complete Linkage Result: Complete linkage produces more compact clusters but tends to over-separate the data, causing digits to be split across clusters. Misclassification remains high.

Average Linkage Result: Average linkage offers moderate performance. It blends cluster distances more smoothly, improving slightly over single and complete linkage, though some digit mixing still occurs.

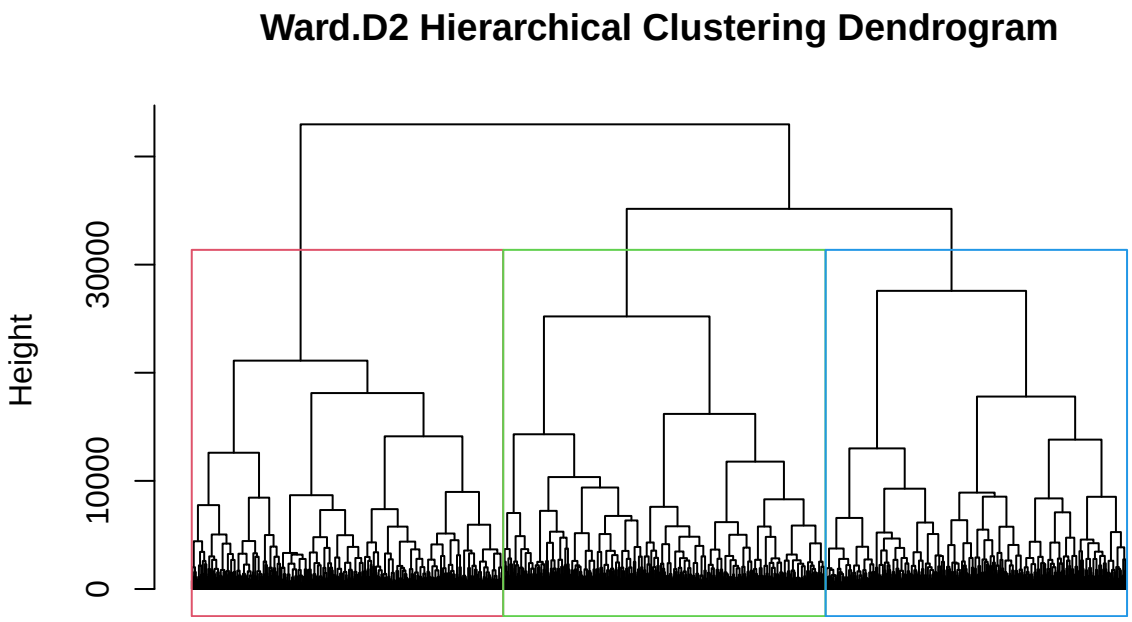
Ward.D2 Result: Ward's method clearly performs best. It works by minimising total within-cluster variance and therefore matches PCA's variance-based structure. This produces well-separated clusters and the lowest misclassification count among all methods.

2.2 Best Method: Ward.D2 (Plots)

2.2.1 Dendrogram

Since Ward.D2 achieved the lowest misclassification count, we examine its clustering structure in more detail. A dendrogram visually represents how observations are merged into clusters based on similarity. By highlighting the 3-cluster division, we can see how clearly Ward's method separates the PCA-reduced data and how tightly the clusters form.

```
# Dendrogram
plot(hc_ward, labels = FALSE, hang = -1, cex = 0.3, sub = "", xlab = "",
     main = "Ward.D2 Hierarchical Clustering Dendrogram")
rect.hclust(hc_ward, k = 3, border = 2:4)
```



The dendrogram shows that Ward's method forms three large, well-defined clusters with relatively large height gaps between branches. These gaps indicate that the algorithm merges points within a cluster before merging across clusters, which is desirable. The drawn rectangles highlight how neatly the data splits into three groups, supporting the earlier finding that Ward.D2 produces the most coherent clustering structure.

2.2.2 Misclassified Points (in red)

To better understand the remaining errors, we highlight the misclassified observations by marking them in red. This reveals where the clustering struggles and which areas of the PCA space contain ambiguous or overlapping digit shapes.

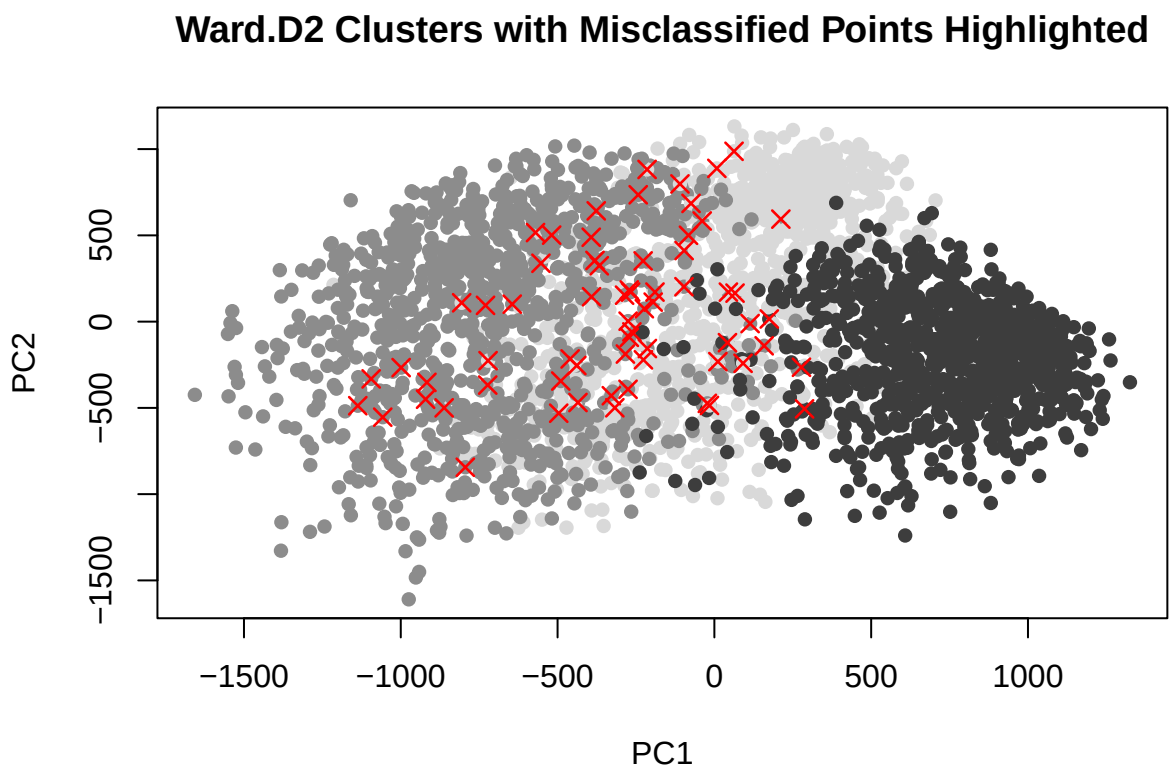
```
gray_cols <- c("#D9D9D9",
               "#8C8C8C",
               "#3D3D3D")

ward_matched <- match_clusters(cut_ward)
cluster_col <- gray_cols[ward_matched]

plot(pca20[,1], pca20[,2],
     col = cluster_col,
     pch = 16,
     xlab = "PC1", ylab = "PC2",
     main = "Ward.D2 Clusters with Misclassified Points Highlighted")

# Misclassified indices
mis_idx <- which(ward_matched != true_label)
```

```r
# Add red X markers
points(pca20[mis_idx,1],
       pca20[mis_idx,2],
       col = "red",
       pch = 4,
       cex = 1.2,
       lwd = 1.3)
```




```r
# Ward linkage confusion matrix
ward_tab <- table(True_Digit = label, Ward_Cluster = cut_ward)
knitr::kable(ward_tab, caption = "Confusion Matrix: Ward.D2 Method")
```

Table 3: Confusion Matrix: Ward.D2 Method

	1	2	3
5	951	46	3
6	12	988	0
7	4	0	996

The misclassified points mostly come from digit 5. Many 5s appear in clusters dominated by digits 6 and 7. This reflects the fact that handwritten 5s in this dataset vary widely in shape—some resemble a rounded 6, while others resemble a slanted 7. In contrast, digits 6 and 7 form very compact and well-separated clusters, with only a few misclassified points each. This shows that Ward.D2 captures the structure of the

data effectively, and the remaining misclassifications arise from ambiguous or inconsistent versions of digit 5.

3 PCA Reconstruction

The purpose of this task is to understand how PCA can be used not only for dimension reduction but also for reconstructing approximate versions of the original images. By keeping only the first few principal components, we can see how much of each digit's structure is preserved at different compression levels.

3.1 Showing the Inverse PCA Transformation

Let X be the data matrix with n observations and d variables.
The covariance matrix of X can be written using the eigen-decomposition

$$\Sigma = U\Lambda U^\top,$$

where
- U contains the eigenvectors (principal directions),
- Λ is the diagonal matrix of eigenvalues.

PCA transforms the original data into principal component (PC) scores using

$$T = XU.$$

Here, T is the matrix of PC scores, and the matrix U gives the rotation from the original coordinate system into the PCA coordinate system.

Because U is an orthogonal matrix, it satisfies

$$U^\top U = I.$$

Multiplying both sides of $T = XU$ by $U^\top$ gives

$$TU^\top = XUU^\top = X.$$

This shows that the inverse PCA transformation is

$$X = TU^\top.$$

So PCA can return to the original data when all components are used.

```r
# Reload data
read_mat <- as.matrix(read.table("../data/digit.txt", sep=","))

image_mat <- read_mat[, 2:785]
label <- read_mat[, 1]

# Perform PCA
pca_out <- prcomp(image_mat, center = TRUE, scale. = FALSE)
scores <- pca_out$x # T matrix (n x 784)
U <- pca_out$rotation # U matrix (784 x 784)
```

The object *scores* stores each image in PCA space, while U stores the basis vectors required to reconstruct the data. These two matrices allow us to perform reduced-order reconstruction.

3.2 Reduced-order Reconstruction Using K PCs

Before reconstructing images, we explore how PCA acts as a compression method. Using only the first K principal components, we build approximations of the original images. Smaller K keeps only coarse structure, while larger K recovers more detail.

```
mu = colMeans(image_mat)

# Function to reconstruct with K PCs
reconstruct_K <- function(K) {
  TK <- scores[, 1:K] # First K PC scores
  UK <- U[, 1:K] # First K eigenvectors
  Xhat <- TK %*% t(UK) # Reconstruction X_hat = TK UK^T
  Xhat <- t(t(Xhat)+mu)
  return(Xhat)
}

# Compute reconstructed datasets
Xhat_1  <- reconstruct_K(1)
Xhat_2  <- reconstruct_K(2)
Xhat_5  <- reconstruct_K(5)
Xhat_10 <- reconstruct_K(10)
```

3.3 Plot Reconstruction for One Sample of Digits 5,6,7

We now select one example of digits 5, 6, and 7 and compare how well they are reconstructed with different numbers of PCs.

```
# Pick one random index for each digit
set.seed(256)
idx5 <- sample(which(label == 5), 1)
idx6 <- sample(which(label == 6), 1)
idx7 <- sample(which(label == 7), 1)

chosen_idx <- c(idx5, idx6, idx7)
chosen_idx
```

```
##   98 1636 2045
##   98 1636 2045
```

```
# Helper function to plot a 28x28 image
plot_digit <- function(vec, main_title) {
  image(1:28, 1:28,
        matrix(vec, nrow = 28)[ , 28:1],
        col = gray(seq(0, 1, 0.02)),
        xlab = "", ylab = "", main = main_title,
        axes = FALSE)
}
```

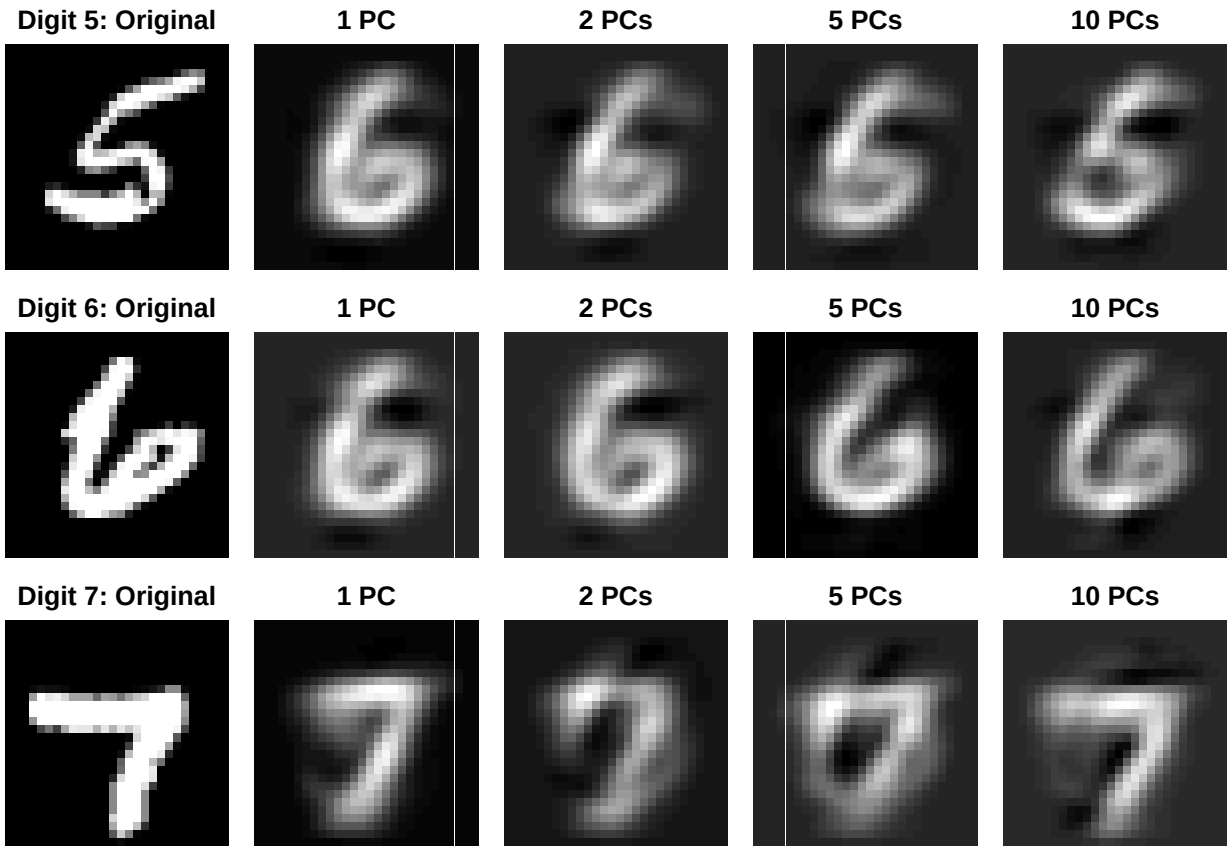
3.4 Plotting Reconstructions for Each Chosen Image

```r
# Set up a grid with 3 rows (digits) and 5 columns (reconstructions)
par(mfrow = c(3, 5), mar = c(0.5, 0.5, 2, 0.5))

# Row 1: Digit 5
plot_digit(image_mat[idx5,], "Digit 5: Original")
plot_digit(Xhat_1[idx5,], "1 PC")
plot_digit(Xhat_2[idx5,], "2 PCs")
plot_digit(Xhat_5[idx5,], "5 PCs")
plot_digit(Xhat_10[idx5,], "10 PCs")

# Row 2: Digit 6
plot_digit(image_mat[idx6,], "Digit 6: Original")
plot_digit(Xhat_1[idx6,], "1 PC")
plot_digit(Xhat_2[idx6,], "2 PCs")
plot_digit(Xhat_5[idx6,], "5 PCs")
plot_digit(Xhat_10[idx6,], "10 PCs")

# Row 3: Digit 7
plot_digit(image_mat[idx7,], "Digit 7: Original")
plot_digit(Xhat_1[idx7,], "1 PC")
plot_digit(Xhat_2[idx7,], "2 PCs")
plot_digit(Xhat_5[idx7,], "5 PCs")
plot_digit(Xhat_10[idx7,], "10 PCs")
```



Across all three digits, the quality of reconstruction improves as the number of principal components increases. With 1 PC, only a faint silhouette is visible. With 2 PCs, the basic structure emerges. With 5 PCs, the digit becomes clearly identifiable. With 10 PCs, most fine details such as bends and line thickness are restored. This shows how PCA compresses the data efficiently while preserving the defining visual features of each handwritten digit.

Conclusion

This analysis demonstrated that Principal Component Analysis (PCA) successfully reduced the complex image data into a compact feature space.

PCA: The scree plot and visualizations confirmed that the first 20 components capture the majority of the variance, effectively filtering out noise.

Clustering: Among the methods tested, Ward's hierarchical clustering achieved the best performance, assigning 97.8% of digits to the correct group without using any labels. This beats K-means (92.1%) and the other linkage methods.

Reconstruction: The inverse transformation showed that while 1-2 PCs capture the general blur, using 10 PCs is sufficient to clearly reconstruct distinct digit shapes.

Overall, the combination of PCA and Ward's method proved to be a robust approach for classifying handwritten digits without supervision.